

The B4X IntelliSense Language Server Protocol Architecture

A Comprehensive Technical Research Paper

Extension: B4X Code IDE Companion (b4x-intellisense)

Version: 0.1.419

Repository: VSCode-B4X-IDE-Companion

Date: June 2026

Table of Contents

1. [Abstract](#)
2. [Introduction](#)
3. [System Architecture Overview](#)
4. [The Extension Host: Client-Side Intelligence](#)
 - 4.1 [Extension Activation & Lifecycle](#)
 - 4.2 [Project File Parsing](#)
 - 4.3 [Platform Discovery & Configuration](#)
 - 4.4 [Library Resolution & Loading](#)
 - 4.5 [The Data Stores](#)
 - 4.6 [Type Inference Engine](#)
 - 4.7 [Language Feature Providers](#)
 - 4.8 [Client-Side Diagnostics](#)
 - 4.9 [Code Actions & Refactoring](#)
5. [The Language Server: Separate-Process Indexing](#)
 - 5.1 [Server Initialization & Capabilities](#)
 - 5.2 [Document Synchronization](#)
 - 5.3 [The Document Manager](#)
 - 5.4 [The Global Symbol Table](#)
 - 5.5 [The Trie Data Structure](#)
 - 5.6 [Worker Pool & Parallel Parsing](#)
 - 5.7 [Symbol Extraction & Parsing](#)
 - 5.8 [LSP Features: Completion, Hover, Definition, Rename](#)

- 5.9 [Server-Side Diagnostics](#)
 - 5.10 [Snapshot Persistence](#)
 - 5.11 [The Extract Method Refactoring](#)
 - 6. [The Communication Protocol](#)
 - 6.1 [Transport Layer](#)
 - 6.2 [Client Startup & Error Recovery](#)
 - 6.3 [Custom Notifications](#)
 - 6.4 [Custom Requests](#)
 - 7. [The Dual-Indexing Architecture](#)
 - 7.1 [Why Two Indexes?](#)
 - 7.2 [Division of Responsibilities](#)
 - 7.3 [How They Complement Each Other](#)
 - 8. [Build & Distribution Architecture](#)
 - 9. [Graceful Degradation & Resilience](#)
 - 10. [Performance Characteristics](#)
 - 11. [Security Considerations](#)
 - 12. [Conclusion](#)
 - 13. [Appendix A: File Reference Map](#)
 - 14. [Appendix B: Data Flow Diagrams](#)
 - 15. [Appendix C: Configuration Reference](#)
-

1. Abstract

This paper presents a thorough architectural analysis of the B4X Code IDE Companion extension for Visual Studio Code — a Language Server Protocol (LSP) implementation that provides IntelliSense, diagnostics, code navigation, refactoring, and type inference for the B4X family of programming languages (B4A for Android, B4i for iOS, B4J for desktop Java, and B4R for Arduino).

The extension employs a **dual-indexing architecture**: a rich, TypeScript-based client running inside VS Code's extension host process that handles deep semantic analysis (type inference, XML library parsing, workspace class indexing, and multi-layer diagnostics), alongside a lightweight Node.js LSP server running in a separate process that provides fast, trie-based symbol completion, hover information, go-to-definition, project-wide rename, and duplicate-symbol diagnostics.

We examine how these two layers communicate via the Language Server Protocol over stdio, how they handle platform-specific library isolation (preventing cross-contamination between B4A, B4i,

B4J, and B4R libraries), how the system gracefully degrades when platform installations or worker threads are unavailable, and how the architecture balances responsiveness against correctness through parallel worker pools, snapshot persistence, and bounded concurrency patterns.

2. Introduction

2.1 What is B4X?

B4X is a family of rapid application development (RAD) tools created by Anywhere Software. It includes:

- **B4A** — Android development (generates Java/Android applications)
- **B4i** — iOS development (generates Objective-C/Swift applications)
- **B4J** — Desktop/server development (generates Java applications)
- **B4R** — Embedded/Arduino development (generates C++/Arduino code)

All four platforms share a BASIC-like syntax and a similar object model. Libraries are distributed as either XML documentation files (which describe classes, methods, properties, and events) or as `.b4xlib` ZIP archives (which contain both the compiled library and its documentation).

2.2 The Challenge

Providing IntelliSense for B4X presents unique challenges:

1. **Platform isolation:** A `.b4a` project must only see B4A libraries, never B4J or B4i libraries — even if all platforms are installed on the same machine.
2. **Multiple library formats:** Libraries can be XML files, `.b4xlib` ZIP archives, or plain `.bas` source modules, each requiring different parsing strategies.
3. **No standard language server:** B4X is a niche language without an existing LSP implementation, so one must be built from scratch.
4. **Heavy type inference:** B4X code uses `Dim` declarations, member access chains (`Button1.Text`), and implicit type resolution that requires cross-referencing library definitions with workspace code.
5. **Windows Registry discovery:** Platform installation paths must be discovered from the Windows Registry, with fallbacks to known paths and user configuration.

2.3 Design Philosophy

The extension follows a principle stated clearly in its architecture comments:

Nothing is assumed. Everything is factual.

Only files confirmed to exist on disk are loaded into IntelliSense. Only libraries explicitly declared in the project file's `LibraryN=` entries are loaded. No cross-platform contamination.

This principle manifests in defensive code throughout: every file path is validated with `fs.stat()` before loading, every library is checked against the project's declared `LibraryN=` list, and every platform lookup falls back through multiple discovery chains before giving up silently.

3. System Architecture Overview

The B4X IntelliSense extension consists of two cooperating processes:

VS Code Extension Host

extension.ts (Orchestrator)

WorkspaceCl
assStore

XmlLibrary
Store

PrimitiveType
Store

CommonClass
Store

TypeInference
Engine

LocalSymbols
Collector

Diagnostics
(4 kinds)

Providers
(15+ kinds)

ProjectFile
Parser

lspClient.ts (Client Bridge)

- Spawns LSP server via vscode-languageclient
- stdio transport (forked child process)
- Registers custom notification handlers
- Forwards b4x/extractMethod requests
- 5-restart budget within 5-minute window

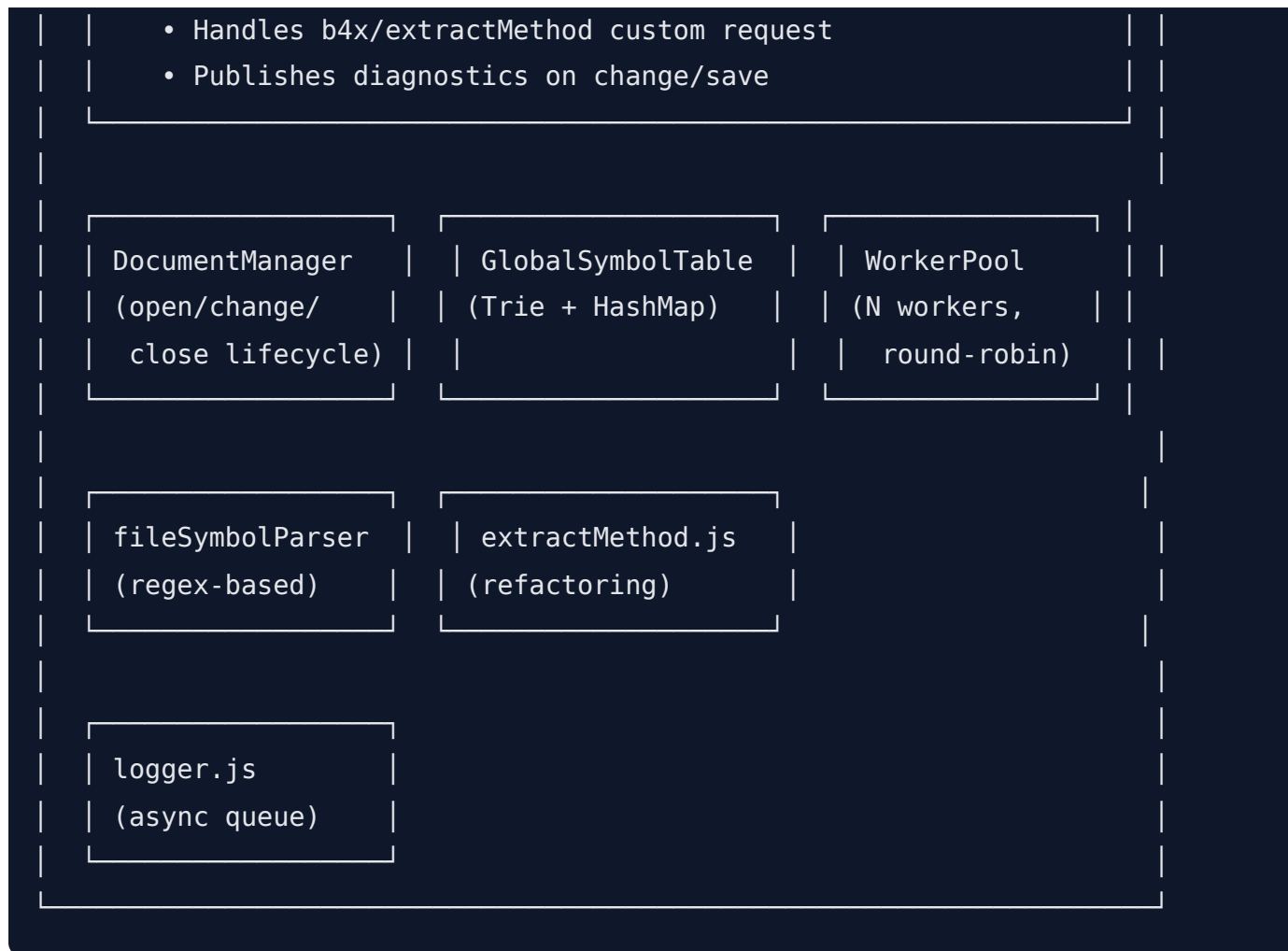
| stdio (fork)



LSP Server (Node.js Process)

server.js (Entry Point)

- Creates connection with ProposedFeatures.all
- Registers document sync (Full), completion, hover, definition, rename handlers



The key insight is that **the client and server have overlapping but distinct responsibilities**:

- The **client** (extension host) has deep semantic understanding: it parses XML libraries, infers types, understands B4X class structures, and provides rich IntelliSense completions with parameter signatures, documentation, and type-aware member suggestions.
- The **server** (separate process) provides fast, project-wide symbol indexing through a trie data structure, enabling instant prefix-based completion, go-to-definition across files, hover previews, and project-wide rename with case-preserving replacements.

4. The Extension Host: Client-Side Intelligence

4.1 Extension Activation & Lifecycle

The extension activates when a B4X project file (`.b4a` , `.b4i` , `.b4j` , `.b4r`) is opened. The `activate()` function in `extension.ts` — at over 4,700 lines, the largest file in the codebase — orchestrates the entire startup sequence:

Step 1: Yield to UI

The function immediately calls `_yieldToUI()` to ensure VS Code renders the "Initializing..." status

bar before the extension begins heavy work.

Step 2: Create Data Stores

Four singleton stores are instantiated:

- `WorkspaceClassStore` — indexes `.bas` / `.b4x` module files into class structures
- `XmlLibraryStore` — parses XML library files into class/method/property definitions
- `PrimitiveTypeStore` — provides synthetic class definitions for primitive types (Int, Float, Double, etc.)
- `CommonClassStore` — extracts the `Common` class from `Core.xml` for bare-word completions (Log, MsgBox, CRLF, etc.)

Step 3: Warm Platform Cache

`warmPlatformCache()` asynchronously discovers B4X installation directories from the Windows Registry, avoiding blocking the UI during the synchronous discovery phase.

Step 4: Register UI Providers

The extension registers webview-based UI components:

- `OllamaChatProvider` — AI chat integration via Ollama
- `CompanionDashboardProvider` — project resources panel
- `LibraryTreeProvider` — sidebar library browser
- `LibraryCatalog` — remote library metadata from GitHub + Google Sheets

Step 5: Initialize SQLite Index

The `libraryIndex.init()` call sets up a persistent SQLite database (using `sql.js` WASM with in-memory fallback) for caching parsed library metadata across sessions.

Step 6: Auto-Detect B4X Projects

If no previous project is found, the extension scans the workspace for `.b4a` / `.b4i` / `.b4j` / `.b4r` files. When exactly one project is found, it opens automatically. When multiple are found, it presents a Quick Pick dialog.

Step 7: Open Project & Load Assets

When the user opens a project (via command or auto-detection), the `openB4xProject` command triggers the full loading pipeline described in Section 4.4.

4.2 Project File Parsing

The `projectFile.ts` module is responsible for extracting project metadata from B4X project files. These files have a unique dual-section format:

```

#Region Project Attributes
Library1=Core
Library2=B4XPages
Module1=|Relative|Main.bas
Module2=|Relative|DBUtils.bas
File1=layout1.bal
#End Region

@EndOfDesignText@

' Code after this marker is the Main module
Sub Process_Globals
    ...
End Sub

```

The parser:

1. **Reads lines before @EndOfDesignText@** — these are project configuration
2. **Extracts LibraryN= entries** → `allowedLibraries` Set (lowercased for case-insensitive matching)
3. **Extracts ModuleN= entries** — strips path prefixes (`|relative|` , `|absolute|` , `|shared|`), normalizes separators, resolves to disk paths
4. **Extracts FileN= entries** — project resource files (layouts, images, etc.)
5. **Generates Main.b4x** — code after `@EndOfDesignText@` is written to `.vscode/b4x-main/<Project>_Main.b4x` for the LSP server to index

The parser also implements **module resolution with fallback chains**:

- First tries the path relative to the project directory
- Then tries each shared module folder
- Missing modules are silently skipped

4.3 Platform Discovery & Configuration

The extension discovers platform installation directories through a three-tier fallback chain:

Tier 1: VS Code Settings

The user can explicitly configure paths:

- `b4xIntellisense.b4aIniPath`
- `b4xIntellisense.b4jIniPath`

- `b4xIntellisense.b4iIniPath`
- `b4xIntellisense.b4rIniPath`

Tier 2: AppData Discovery

Default INI file locations per platform:

- B4A: `%APPDATA%\Anywhere Software\Basic4android\b4xV5.ini`
- B4J: `%APPDATA%\Anywhere Software\B4J\b4xV5.ini`
- B4i: `%APPDATA%\Anywhere Software\B4i\b4xV5.ini`
- B4R: `%APPDATA%\Anywhere Software\B4R\b4xV5.ini`

Tier 3: Windows Registry

The `findPlatformInstallDirs()` function queries the Windows Registry via PowerShell, searching `HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall` and `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall` for entries matching "B4A", "B4J", "B4R", "B4I" with `InstallLocation` values.

The platform INI files (`b4xV5.ini`) contain four critical folder paths:

- `LibrariesFolder` — the platform's internal library directory
- `AdditionalLibrariesFolder` — user-installed additional libraries
- `SharedModuleFolder` — shared `.bas` modules across projects
- `PlatformFolder` — SDK platform path (e.g., `android-36`)

4.4 Library Resolution & Loading

Once the platform INI is parsed, the `reloadPlatformAssets()` function in `extension.ts` resolves libraries through a strict chain:

For each `LibraryN=` entry in the project file:

1. Search: `LibrariesFolder/<lib>.xml`
2. Search: `LibrariesFolder/<lib>/<lib>.xml`
3. Search: `AdditionalLibrariesFolder/<lib>.xml`
4. Search: `AdditionalLibrariesFolder/<lib>/<lib>.xml`
5. Search: `LibrariesFolder/<lib>.b4xlib` (only if no XML found)
6. Search: `AdditionalLibrariesFolder/<lib>.b4xlib` (only if no XML found)
7. `fs.stat()` confirms file exists – MISSING FILES LOGGED AND SKIPPED

This is the critical **platform isolation mechanism**: a `.b4a` project only searches B4A's libraries folder, never B4J's or B4i's.

For `.b4xlib` files, the extension:

1. Opens the ZIP archive using `node-stream-zip`
2. Extracts `.bas` module files to a cache directory
3. Reads `manifest.txt` for version and dependency information
4. Registers extracted modules as reference modules in `WorkspaceClassStore`

The loading order matters for priority:

- **Workspace modules** (from `ModuleN=` entries) take highest priority
- **Reference modules** (from `.b4xlib` extraction) have second priority
- **XML library classes** have lowest priority
- In all cases, **the first-loaded definition wins** — later definitions for the same class name are discarded

4.5 The Data Stores

`WorkspaceClassStore` (`workspaceClassIndex.ts`)

This store parses `.bas` module files into structured class definitions. It understands three module types:

Module Type	Detection Criteria	Characteristics
Class	<code>Type=Class</code> header + <code>Sub Class_Globals</code>	Has global properties, methods, events
Static	<code>Type=StaticCode</code> header + <code>Sub Process_Globals</code>	Has global properties but no events
Service	<code>Sub Service_Create</code> + <code>Sub Service_Start</code> or <code>#StartAtBoot:</code>	Service lifecycle module

The parser extracts:

- **Methods** — `Public Sub Name(params) As ReturnType`, including visibility, parameters with types, and return types
- **Properties** — From `Dim / Public / Private` declarations inside `Class_Globals` or `Process_Globals`, plus inferred getter/setter pairs (`GetValue / SetValue` → property `Value`)
- **Events** — `#Event: EventName(params)` directives
- **Type fields** — `Type Name(field1 As Type1, ...)` declarations inside globals blocks

- **Leading comments** — Comment lines before the first code line become class-level documentation

A key optimization is the `LightweightDocument` adapter that avoids VS Code's heavyweight document lifecycle (language detection, model creation, event emission) when the extension only needs to read file text for parsing.

XmlLibraryStore (`xmlLibraryIndex.ts`)

This store parses B4X XML library files (like `Core.xml` , `SQL.xml`) into class definitions. XML libraries define classes with:

- `<method>` elements → method definitions with parameters and return types
- `<property>` elements → property definitions with access modifiers
- `<field>` elements → field definitions (constants, type fields)
- `<event>` elements → event definitions with parameter signatures
- `<version>` elements → library version information

The parser handles the full B4X XML format including documentation extraction, short-name derivation, and method/property merging.

CommonClassStore (`commonClassStore.ts`)

In B4X, the `Common` class (defined in `Core.xml`) contains global functions that can be called without a class prefix: `Log()` , `Msgbox()` , `CRLF` , `TAB` , etc. This store extracts the `Common` class and provides these as bare-word completions.

PrimitiveTypeStore (`primitiveTypeStore.ts`)

Provides synthetic class definitions for B4X primitive types (`Int` , `Float` , `Double` , `Long` , `Boolean` , etc.) that don't have explicit XML class entries. Also maintains a type mapping for resolving type aliases (e.g., `String2` → `String` in B4A).

4.6 Type Inference Engine

The type inference system (`b4xTypeInference.ts`) is one of the most sophisticated parts of the client-side intelligence. It enables dot-completion on member access expressions like:

```
Dim btn As Button  
btn.T|    ← completes to btn.Text, btn.Tag, etc.
```

The engine works in three phases:

Phase 1: Local Variable Collection

`inferVariableTypes()` scans the current document for `Dim` declarations and `Sub` parameter signatures, building a `Map<string, string>` of variable names to their declared types.

Phase 2: Expression Resolution

When the user types a dot-access expression like `btn.Text.Length`, `resolveExpressionType()` chains through:

1. `btn` → looks up in local variables → `Button`
2. `Button` → looks up in workspace classes and XML libraries → finds the `Button` class
3. `.Text` → looks up `Text` property in `Button` → `String`
4. `.Length` → looks up `Length` property in `String` → `Int`

Phase 3: Owner Class Inference

`inferCompletionOwnerClass()` determines which class to show completions for, using:

1. The member access expression prefix (`btn.T` → `btn`)
2. The resolved type of the expression (`btn` → `Button`)
3. The matching class definition from the data stores

4.7 Language Feature Providers

The extension registers 15+ VS Code language feature providers, all operating on the B4X language ID:

Provider	Source	Purpose
CompletionItemProvider	Client-side (stores + inference)	Context-aware completions with type inference
HoverProvider	Client-side + LSP server	Shows definition source code on hover
DefinitionProvider	LSP server	Go to definition across project files
ReferenceProvider	Client-side	Find all references across workspace
RenameProvider	LSP server	Project-wide rename with case preservation
FoldingRangeProvider	Client-side	Code folding for Sub/Type/If/For blocks

DocumentSymbolProvider	Client-side	Outline view (Sub/Type/Class symbols)
WorkspaceSymbolProvider	Client-side	Workspace-wide symbol search
DocumentFormattingProvider	Client-side	Full document formatting
DocumentRangeFormattingProvider	Client-side	Selection formatting
OnTypeFormattingProvider	Client-side	Auto-formatting on typing
DocumentHighlightProvider	Client-side	Highlight occurrences under cursor
DocumentLinkProvider	Client-side	Clickable links in code
SelectionRangeProvider	Client-side	Expand/shrink selection
CodeLensProvider	Client-side	Reference counts above Subs
SignatureHelpProvider	Client-side	Parameter hints for Sub calls
InlineCompletionProvider	Client-side	Ghost text completions
ImplementationProvider	Client-side	Go to Implementation
TypeDefinitionProvider	Client-side	Go to Type Definition

The **CompletionItemProvider** is the most complex, merging results from:

1. B4X keywords and directives
2. Common class bare-word completions
3. Workspace class Subs, properties, and events
4. XML library class methods and properties
5. Type-inferred member completions
6. Local variable completions
7. Primitive type completions

4.8 Client-Side Diagnostics

The extension provides four client-side diagnostic providers, each following the same pattern: a pure computation function returns `Diagnostic[]`, and a registration function creates a `DiagnosticCollection` that subscribes to document events.

Type Diagnostics (`typeDiagnostics.ts`)

Warns when `Type` declarations appear outside `Sub Class_Globals` or `Sub Process_Globals`. In B4X, custom types must be declared in global scope, and this diagnostic catches violations early.

CallSub Validation (`callSubDiagnostics.ts`)

Validates that `CallSub("Module", "SubName")` and `CallSubDelayed("Module", "SubName")` targets actually exist in the workspace. This catches a common B4X bug where a Sub is renamed but the string-based CallSub reference isn't updated.

Unused Sub Detection (`unusedSubDiagnostics.ts`)

Detects Sub routines that are never called anywhere in the workspace. It uses sophisticated exclusion logic:

- **Event handlers** are excluded (e.g., `Button1_Click`, `Activity_Create`)
- **Lifecycle Subs** are excluded (e.g., `Activity_Create`, `Service_Start`)
- **Private Subs** are excluded (they may be called from the same module)
- **B4X lifecycle Subs** (`_Create`, `_Start`, `_Pause`, etc.) are always excluded

Unused Library Detection (`unusedLibraryDiagnostics.ts`)

Cross-references declared libraries (from `LibraryN=` entries) against actually-used types in workspace code. If a library is declared but none of its types are referenced, it flags a warning suggesting the library may be unnecessary.

4.9 Code Actions & Refactoring

Type Code Actions (`typeCodeAction.ts`)

Provides quick fixes for Type diagnostics — suggesting to move `Type` declarations into `Class_Globals` or `Process_Globals`.

Extract Method (`extractMethodCodeAction.ts`, `server/indexer/extractMethod.js`)

The Extract Method refactoring is implemented as a collaboration between client and server:

1. The client-side `ExtractMethodCodeActionProvider` detects when the user selects code and offers "Extract Method"
2. When invoked, it sends a `b4x/extractMethod` request to the LSP server
3. The server analyzes the selected code to:
 - Identify identifiers used in the selection but declared outside it (these become parameters)
 - Score candidates based on proximity to the selection
 - Create the new Sub with inferred parameters
 - Replace the selection with a call to the new Sub

The parameter inference algorithm:

1. Collects all identifiers in the selected code
 2. Identifies which are declared inside the selection (local variables → not parameters)
 3. Identifies which are declared outside the selection in the same Sub (potential parameters)
 4. Identifies which are in `Class_Globals` or `Process_Globals` (not needed as parameters)
 5. Scores each candidate: +20 if declared outside selection, +10 if used before selection, +8 if used after selection, -5 if a global
 6. Filters out B4X keywords and single-character identifiers
 7. Returns the scored and sorted parameter list
-

5. The Language Server: Separate-Process Indexing

5.1 Server Initialization & Capabilities

The LSP server (`server.js`) starts when the client forks a new Node.js process. It immediately creates a `vscode-languageserver` connection with `ProposedFeatures.all` to enable the latest protocol features.

On `initialize`, the server:

1. Extracts `projectRoot` from `initializationOptions` — this is the B4X project directory
2. Starts async disk indexing via `docManager.loadFromDisk(root, connection, workerPool)`
3. Returns capabilities:

```
{
  "textDocumentSync": "Full",
  "completionProvider": { "resolveProvider": false },
  "hoverProvider": true,
  "definitionProvider": true,
  "renameProvider": { "prepareProvider": true }
}
```

The `Full` sync mode means the client sends the entire document content on every change. This simplifies the server — it always has the complete file text — at the cost of bandwidth. For B4X files, which are typically small, this is an acceptable trade-off.

5.2 Document Synchronization

The server uses `vscode-languageserver`'s `TextDocuments` manager with `TextDocumentSyncKind.Full`. The document lifecycle events are:

`onDidChangeContent` — Primary handler. Fires when a document's text changes. The server:

1. Clears any blacklisted file status for the URI (allowing re-indexing after a previous worker crash)
2. Calls `docManager.changeDocument(uri, text)` which re-parses the document's symbols
3. Calls `publishDiagnosticsForUri(uri)` which checks for duplicate symbols and misplaced Type declarations

`onDidOpen` — Fires when a document is first opened. Skips if the document is already tracked (because `onDidChangeContent` fires immediately after with the same text, and the `DocumentManager` checks for duplicates).

`onDidClose` — Removes the document from the `DocumentManager` and its symbols from the `GlobalSymbolTable`.

`onDidSave` — Re-parses the document using the `WorkerPool` for parallel processing. Uses a monotonic sequence counter (`parseSequence`) to discard stale results: if a newer save has occurred since the parse was submitted, the old results are discarded.

5.3 The Document Manager

`DocumentManager` (`server/indexer/documentManager.js`) is the central coordinator for document state in the LSP server. It maintains:

- `docs` — A `Map<string, {text: string, symbols: Array}>` tracking every open document's text and parsed symbols
- `global` — A `GlobalSymbolTable` instance for cross-file symbol lookups

Key methods:

Method	Purpose
<code>openDocument(uri, text)</code>	Parse file, add to docs map and global table
<code>changeDocument(uri, text)</code>	Re-parse file, update docs map and global table
<code>closeDocument(uri)</code>	Remove from docs map and global table
<code>setSymbolsForUri(uri, symbols)</code>	Update symbols for a document (from worker results)
<code>getCompletions(prefix)</code>	Delegate to <code>GlobalSymbolTable.getByPrefix(prefix, 100)</code>

<code>findDefinition(word)</code>	Delegate to <code>GlobalSymbolTable.getByExactName(word)</code>
<code>loadFromDisk(root, connection, workerPool)</code>	Walk directory for <code>.bas</code> / <code>.b4x</code> files, parse them in parallel
<code>saveSnapshot(root)</code>	Persist global symbol table to <code>.b4x-index.json</code>
<code>loadSnapshot(root)</code>	Restore global symbol table from <code>.b4x-index.json</code>

The `loadFromDisk` method is particularly important for understanding the startup flow:

1. **Snapshot first:** Loads `.b4x-index.json` if available for instant basic IntelliSense
2. **Directory walk:** Recursively walks the project directory, skipping `node_modules` and `.git`
3. **Parallel parsing:** Processes files with bounded concurrency (8 concurrent) using the `WorkerPool`
4. **Progress notifications:** Sends `b4x/indexing` notifications to the client at start, every 20 files or every 1 second, and at completion

5.4 The Global Symbol Table

`GlobalSymbolTable` (`server/indexer/globalSymbolTable.js`) is the core data structure for cross-file symbol resolution. It maintains four data structures:

Data Structure	Type	Purpose
<code>byName</code>	<code>Map<string, Symbol[]></code>	Exact name lookup (case-insensitive key)
<code>_trie</code>	<code>TrieNode</code>	Prefix-based lookup for completions
<code>_symbolStore</code>	<code>Map<number, Symbol></code>	ID-to-symbol mapping for deduplication
<code>_fileToIds</code>	<code>Map<string, Set<number>></code>	File-to-symbol-IDs mapping for incremental updates

Each symbol has the shape: `{ kind: 'sub'|'type'|'class', name: string, line: number, file: string, _id: number }`

The key operation is `applyFileSymbols(fileSymbols, filePath)` which performs incremental updates:

1. **Remove old symbols** for the file from `byName`, `_trie`, and `_symbolStore`
2. **Add new symbols** to `byName`, `_trie`, and `_symbolStore`

3. Update `_fileToIds` mapping

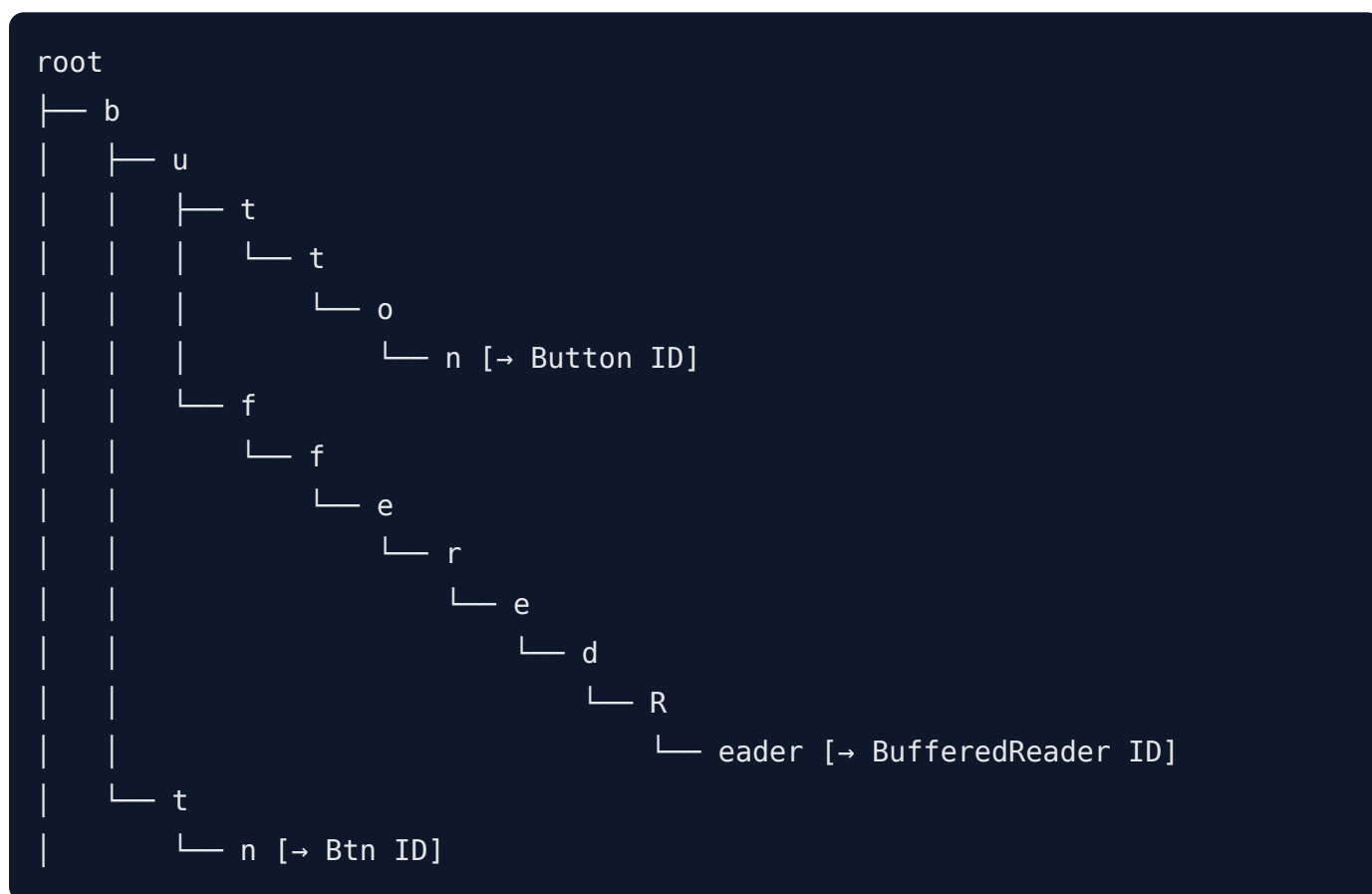
This incremental approach means updating a file's symbols only touches the symbols that changed, not the entire table. When a file is closed, `removeFile(filePath)` removes all its symbols.

5.5 The Trie Data Structure

The trie (`GlobalSymbolTable._trie`) enables $O(\text{prefix-length})$ prefix-based completion lookups. Each node has:

- `children` — A `Map<string, TrieNode>` for child characters
- `symbolIds` — A `Set<number>` of symbol IDs stored at this leaf

For example, with symbols `Button`, `Btn`, `BufferedReader`, the trie looks like:



`getByPrefix("bu")` would walk `b → u` and collect all symbols below, returning `Button` and `BufferedReader` in $O(2)$ lookup time.

The `serialize/deserialize` methods persist the entire symbol table (including the `byName` map and `_symbolStore`) as JSON, allowing instant startup on subsequent loads.

5.6 Worker Pool & Parallel Parsing

The `WorkerPool` (`server/indexer/workerPool.js`) provides parallel file parsing using Node.js `worker_threads`:

Configuration:

- Pool size: `min(2, cpus - 1)` — capped at 2 workers to limit memory usage
- Parse timeout: 30 seconds per request
- Round-robin distribution via `nextWorker` counter

Fault tolerance:

- **Blacklisting:** Files that crash a worker are added to `failedFiles` and skipped on future parse attempts. When a document changes (via `onDidChangeContent`), the file is removed from the blacklist, giving it another chance.
- **Worker recreation:** If a worker exits with an error code, it's recreated (but only if workers were previously available — in sandboxed environments where no workers can be created, recreation would loop forever).
- **Sandbox degradation:** If no workers can be created (e.g., in VS Code's restricted mode), the pool gracefully returns rejection Promises. Callers in `DocumentManager.loadFromDisk()` catch these errors and skip the file, meaning indexing is degraded but not broken.

The `workerTask.js` entry point receives `{id, uri, text}` messages, calls `parseFile(text, filePath)`, and posts back `{id, symbols, uri}` or `{id, error}`.

5.7 Symbol Extraction & Parsing

`fileSymbolParser.js` is intentionally minimal — a heuristic regex-based parser that extracts only three kinds of symbols:

```
const subRegex    = /^\\s*Sub\\s+([A-Za-z_][A-Za-z0-9_]*)/i;
const typeRegex   = /^\\s*Type\\s+([A-Za-z_][A-Za-z0-9_]*)/i;
const classRegex  = /^\\s*Sub\\s+Class_?\\s*([A-Za-z_][A-Za-z0-9_]*)/i;
```

This returns symbols of shape: `{ kind: 'sub'|'type'|'class', name: string, line: number, file: string }`

The `classRegex` matches the B4X pattern `Sub Class_Globals` (and variations) as class declarations. The `subRegex` catches all Sub declarations including event handlers.

This simplicity is deliberate: the LSP server's job is **fast, project-wide indexing**, not deep semantic analysis. The client-side TypeScript code handles the rich parsing (type inference, method signatures, etc.).

5.8 LSP Features: Completion, Hover, Definition, Rename

Completion (`onCompletion`)

1. Extract the word prefix at the cursor position
2. Call `docManager.getCompletions(prefix)` which uses the trie-based prefix search
3. Return up to 100 completion items with label, kind (always `3` = Function), and detail string showing `kind – file:line`

The detail format like `sub – Main.bas:42` allows the user to distinguish between multiple symbols with the same name.

Hover (`onHover`)

1. Extract the word at the cursor position
2. Call `docManager.findDefinition(word)` which uses the exact-name map
3. If found, read the source file at the definition location and return 5 lines of context (2 before, the definition line, 2 after) as a markdown code block
4. If not found, return a fallback "LSP scaffold running." message

Definition (`onDefinition`)

1. Extract the word at the cursor position
2. Call `docManager.findDefinition(word)` for exact name lookup
3. Convert the definition's file path to a `file://` URI
4. Return a `Location` with the range spanning from column 0 to the end of the symbol name

Rename (`onRenameRequest`)

The rename implementation is notably sophisticated:

1. Extract the old name at the cursor position
2. Look up the definition to confirm the symbol exists
3. Collect **candidate file paths** from:
 - All open documents in the DocumentManager
 - All files containing the same symbol name (from `global.getByExactName`)
 - The definition file itself
4. Process files with bounded concurrency (4 files at a time) to avoid memory spikes
5. For each file, search for all occurrences of the old name using a word-boundary regex
6. **Skip occurrences inside quoted strings and comments** using two helper functions:

- `isInQuotedString()` — counts unescaped double quotes to determine if a position is inside a string literal
- `isCommentLine()` — checks if a `'` or `//` comment marker appears before the match position

7. **Preserve case** — if the original occurrence is `MYVAR`, the replacement is `NEWVAR`; if `MyVar`, then `NewVar`; if `myvar`, then `newvar`

This ensures that renaming a Sub across a project doesn't accidentally rename string literals or comments containing the same word, and preserves the original casing convention.

5.9 Server-Side Diagnostics

The LSP server publishes two kinds of diagnostics:

Duplicate Symbol Warnings

For each symbol in a file, checks if the same name exists in other files. If so, emits a warning:

```
"Symbol 'X' is also defined in other files (file1.bas, file2.bas)"
```

Misplaced Type Errors

For each `Type` declaration, checks if a `Sub Class_Globals` or `Sub Process_Globals` line appears within 6 lines before it. If not, emits an error:

```
"Type 'X' appears outside Class_Globals/Process_Globals (heuristic)"
```

Both diagnostics use a 6-line lookback heuristic that balances accuracy with performance.

5.10 Snapshot Persistence

On server shutdown, `docManager.saveSnapshot(workspaceRoot)` persists the entire `GlobalSymbolTable` to `.b4x-index.json` in the project root. This file contains:

- The `_symbolStore` map (symbol ID → symbol data)
- The `_fileToIds` map (file path → symbol IDs)
- The `_symbolId` counter (for generating new IDs)

On next startup, `loadSnapshot()` deserializes this file and rebuilds the `byName` map and `_trie` from the stored data, providing **instant basic IntelliSense** before the full disk scan completes. The full scan then updates the table with any changes since the snapshot was saved.

The snapshot file is intentionally simple JSON — no binary format, no compression — because:

- It's small (typically a few hundred KB for a medium project)

- It needs to be loadable by a simple `JSON.parse()` without special dependencies
- It's human-readable for debugging

5.11 The Extract Method Refactoring

The `extractMethod.js` module creates a `WorkspaceEdit` for the "Extract Method" refactoring:

1. Given a file path, selection range, new method name, and optional parameters
2. Extracts the selected lines into a new `Sub NewName(params) ... End Sub` at the end of the file
3. Replaces the original selection with a call to the new `Sub`
4. Uses B4X's no-parentheses call syntax when there are parameters: `NewName param1, param2` instead of `NewName(param1, param2)`

The parameter inference (performed on the server side in the `b4x/extractMethod` request handler) is described in Section 4.9.

6. The Communication Protocol

6.1 Transport Layer

The LSP client and server communicate via **stdio** (standard input/output). The client forks the server process with `TransportKind.stdio`, which means:

- **stdin** — Client → Server (JSON-RPC requests and notifications)
- **stdout** — Server → Client (JSON-RPC responses and notifications)
- **stderr** — Available for logging (the server logger writes to `server/logs/server.log`)

In debug mode, the server is launched with `--inspect=127.0.0.1:6009` for Node.js debugger attachment.

6.2 Client Startup & Error Recovery

The `lspClient.ts` module implements a robust startup and recovery protocol:

Server Module Verification

Before forking the server process, the client verifies that `dist/server.js` exists using `fs.access()`. This prevents silent spawn failures in sandboxed environments.

Error Handling

The `errorHandler` configuration uses:

- `ErrorAction.Continue` (2) — transient errors don't shut down the client

- `CloseAction.Restart` (2) — the server is restarted on close, with a budget of 5 restarts within 5 minutes
- After exceeding 5 restarts: `CloseAction.DoNotRestart` (1) — stops trying and shows an error message

Sandbox Detection

If the server fails to start with an `EACCES`, `EPERM`, or `ENOENT` error, or if the error message contains "spawn" or "fork" (but not "ForkJoin"), the client shows a specific message:

"B4X: Language server failed to start — the sandboxed environment may be blocking process creation. Try trusting the workspace and reloading."

This was added to handle VS Code's restricted mode, where workspace trust is required before child processes can be spawned.

6.3 Custom Notifications

b4x/indexing (Server → Client)

Sent during disk indexing with phases:

- `start` — { `phase`: "start", `total`: N }
- `progress` — { `phase`: "progress", `processed`: M, `total`: N } (sent every 20 files or every 1 second)
- `done` — { `phase`: "done", `processed`: N, `total`: N }

The client uses these to update the VS Code status bar with "Indexing workspace (M/N)" messages.

b4x/indexingFailed (Server → Client)

Sent if disk indexing throws an error. The client shows an error message to the user.

6.4 Custom Requests

b4x/extractMethod (Client → Server)

Parameters: { `uri`: string, `range`: { `start`: Position, `end`: Position }, `newName?`: string, `params?`: string[] }

Returns: { `changes`: { [uri: string]: TextEdit[] }, `suggestedParams`: { `name`: string, `type`: string }[] }

This is the only custom LSP request. It's triggered by the Extract Method code action in the client.

7. The Dual-Indexing Architecture

7.1 Why Two Indexes?

The most distinctive aspect of this LSP implementation is that **there are two independent symbol indexes** — one in the extension host (TypeScript) and one in the LSP server (JavaScript):

Feature	Client (Extension Host)	Server (LSP Process)
Language	TypeScript	JavaScript
Parsing depth	Deep (Sub signatures, types, events, properties)	Shallow (Sub/Type/Class names only)
Data structures	HashMap by class name	Trie + HashMap by symbol name
Lookup mode	Exact name (for a specific class)	Prefix-based (for completions)
Libraries	Full XML parsing with methods, properties, docs	Not loaded
Workspace modules	Full <code>.bas</code> parsing with visibility, events, types	Name-only parsing
Type inference	Full expression chaining	Not supported
Scope	Current project + declared libraries	Current project files only

7.2 Division of Responsibilities

Client handles:

- All completion items (merges keywords, library classes, workspace classes, type inference)
- Hover that shows full method signatures with documentation
- Signature help (parameter hints)
- Find All References (scans all workspace files)
- Folding ranges, document symbols, workspace symbols
- Document formatting (indentation rules)
- Diagnostics (type placement, CallSub targets, unused Subs, unused libraries)
- Code actions (quick fixes)

Server handles:

- Cross-file go-to-definition (via the global symbol table)
- Cross-file rename (project-wide, with case preservation and comment/string exclusion)
- Fast prefix-based completion (via the trie)

- Hover with source code context
- Duplicate symbol diagnostics
- Extract Method refactoring

7.3 How They Complement Each Other

The client's deep parsing provides rich completions with full type information, but it doesn't easily handle cross-file operations like "find all references to Sub XYZ across the entire project." The server's trie-based index excels at exactly this: by indexing every symbol in every `.bas` / `.b4x` file, it can instantly find definitions, complete prefixes, and rename across the entire project.

The two indexes are **not synchronized** — they operate independently. The client loads its data when a project is opened and refreshes on document changes. The server loads its data from disk on startup and updates on document changes. This means there can be brief inconsistencies (e.g., a newly created Sub might appear in client-side completions before the server has re-indexed the file), but in practice these are barely noticeable because both indexes update on the same document change events.

8. Build & Distribution Architecture

The build system uses **esbuild** (via `scripts/bundle.js`) to produce three bundles:

Bundle	Entry	Output	Externals	Purpose
Extension	<code>src/extension.ts</code>	<code>dist/extension.js</code>	<code>vscode</code>	VS Code extension host code
Server	<code>server/server.js</code>	<code>dist/server.js</code>	None	LSP server (all deps inlined)
Worker	<code>server/indexer/workerTask.js</code>	<code>dist/workerTask.js</code>	None	Worker thread for parallel parsing

The server and worker bundles inline all dependencies (including `vscode-languageserver`, `vscode-languageserver-textdocument`, etc.) so the LSP process doesn't need `node_modules` at runtime. This was a critical fix — the previous approach of shipping unbundled code caused

"Cannot find module" errors when VS Code's ASAR packing made `node_modules/` inaccessible to the forked server process.

The build also copies `sql-wasm.wasm` (for the SQLite persistence layer) to `dist/`.

The `package.json` entry point is `./dist/extension.js`, which is switched between bundled and development paths by the build scripts.

9. Graceful Degradation & Resilience

The extension is designed to degrade gracefully when components are unavailable:

No platform installation found → Zero libraries loaded, but the extension still provides keyword completions and basic Sub/Type/Class indexing from workspace files.

No worker threads available (sandboxed environment) → The WorkerPool returns rejection Promises, and `DocumentManager.loadFromDisk()` catches these errors and skips the file. Single-threaded parsing via `fileSymbolParser.js` is used for open documents.

Snapshot file missing or corrupt → `loadSnapshot()` silently catches errors and proceeds with an empty table. The full disk scan rebuilds it.

Library file missing → Logged as a warning and skipped. No error thrown to the user.

Module file not found → Silently skipped. The module won't appear in IntelliSense.

LSP server crash → The client restarts up to 5 times within 5 minutes. After that, it shows an error message and stops trying.

Worker crash during parsing → The file is blacklisted. On next document change, the blacklist entry is cleared and the file gets another chance.

10. Performance Characteristics

Operation	Time Complexity	Notes
Completion lookup (server)	$O(\text{prefix-length})$	Trie traversal, typically < 1ms
Exact name lookup (server)	$O(1)$ amortized	HashMap lookup
Full project index (server)	$O(\text{files} \times \text{avg_symbols})$	Parallelized across 2 workers
Snapshot load	$O(\text{total_symbols})$	Single <code>JSON.parse</code> + trie rebuild

Client-side completion	$O(\text{classes} \times \text{methods})$	Linear scan with prefix matching
Type inference	$O(\text{document_lines} \times \text{depth})$	Single pass + member lookup
Rename (server)	$O(\text{files} \times \text{file_size})$	Bounded to 4 concurrent file reads

The server's trie gives $O(\text{prefix-length})$ completion lookups, which is effectively $O(1)$ for short prefixes and $O(\log n)$ for the longest ones. The client-side completions use linear scans over the loaded classes, which is acceptable for the typical B4X project size (hundreds of classes, thousands of methods).

11. Security Considerations

The extension handles several security-relevant areas:

Registry Access: Platform discovery reads from the Windows Registry using PowerShell. The script is a hardcoded string (no user input injection) and uses `execFileSync` with argument arrays.

File System Access: All file paths are validated with `fs.stat()` before loading. The directory walker skips `node_modules` and `.git`. Symlink cycles are detected using `fs.realpath()` and a `visitedRealPaths` set.

Process Spawning: The LSP server is spawned with `TransportKind.stdio` using `context.asAbsolutePath()` to resolve the module path, preventing path injection.

Worker Isolation: Worker threads are isolated Node.js `worker_threads` — they can only communicate via `postMessage()` and cannot access the main thread's memory or the VS Code API.

Input Validation: The `fileSymbolParser.js` uses strict regex patterns that only match well-formed B4X declarations. The `projectFile.ts` strips BOM characters and normalizes line endings before parsing.

12. Conclusion

The B4X IntelliSense extension demonstrates an effective approach to providing language support for a niche language in VS Code: combining a **rich client-side engine** (for deep semantic analysis, type inference, and platform-aware library resolution) with a **lightweight LSP server** (for fast, project-wide symbol indexing and cross-file operations).

The dual-indexing architecture is the most distinctive design choice. Rather than trying to make a single index serve all purposes, the extension recognizes that different features have different requirements: completions need rich type information (provided by the client's deep parsing), while go-to-definition and rename need fast cross-file symbol lookup (provided by the server's trie index). This separation allows each component to be optimized for its specific use case.

The extension's resilience patterns — graceful degradation, snapshot persistence, worker crash recovery, and bounded restart budgets — reflect hard-won experience with production failures in sandboxed environments, ASAR-packed extensions, and missing platform installations. Every path through the code has a fallback, and every error is caught and handled without crashing the extension or the LSP server.

For developers looking to build a similar extension for another language, the key takeaways are:

1. **Start with a thin LSP server** that handles cross-file operations, and build the rich client-side analysis alongside it
2. **Use a trie for completions** — it provides $O(\text{prefix-length})$ lookups and is trivially updatable
3. **Persist snapshots** for instant startup, then update incrementally
4. **Graceful degradation is not optional** — platforms will be missing, files will be inaccessible, and workers will crash
5. **Platform isolation must be enforced at every level** — library resolution, symbol lookup, and completion filtering must all respect the current platform
6. **The Nothing is assumed principle** prevents entire categories of bugs — if a file doesn't exist on disk, it's simply not loaded, rather than causing a crash or incorrect behavior

Appendix A: File Reference Map

File	Lines	Purpose
src/extension.ts	~4700	Main orchestrator — activation, project loading, provider registration
src/lspClient.ts	~177	LSP client — spawns server, handles notifications, restart budget
src/projectFile.ts	~443	Parses .b4a / .b4i / .b4j / .b4r project files
src/platformConfig.ts	~200	Discovers B4X install paths from Registry and settings

<code>src/platformIni.ts</code>	~200	Parses <code>b4xV5.ini</code> for library/module folder paths
<code>src/workspaceClassIndex.ts</code>	~860	Parses <code>.bas</code> modules into class structures
<code>src/xmlLibraryIndex.ts</code>	~600+	Parses XML library files into class/method/property data
<code>src/commonClassStore.ts</code>	~80	Extracts Common class for bare-word completions
<code>src/primitiveTypeStore.ts</code>	~200	Synthetic class definitions for primitive types
<code>src/b4xTypeInference.ts</code>	~300+	Type inference from Dim declarations and member access
<code>src/b4xLocalSymbols.ts</code>	~120	Collects local symbols (Subs, Types, variables)
<code>src/b4xDocParser.ts</code>	~500+	Core B4X parser (comments, member access, typed names)
<code>src/types.ts</code>	~140	Shared type definitions (B4xClass, B4xMethod, etc.)
<code>src/typeDiagnostics.ts</code>	~80	Warns about Type declarations outside globals
<code>src/callSubDiagnostics.ts</code>	~100	Validates CallSub/CallSubDelayed targets
<code>src/unusedSubDiagnostics.ts</code>	~150	Detects unused Subroutines
<code>src/unusedLibraryDiagnostics.ts</code>	~100	Detects unused libraries
<code>src/extractMethodCodeAction.ts</code>	~80	Client-side Extract Method code action
<code>src/lightweightDocument.ts</code>	~100	Lightweight document adapter (avoids VS Code overhead)
<code>src/storage/libraryIndexSqlite.ts</code>	~300+	SQLite-based persistent library index
<code>server/server.js</code>	~380	LSP server entry point
<code>server/indexer/documentManager.js</code>	~240	Document lifecycle management
<code>server/indexer/globalSymbolTable.js</code>	~185	Trie-based symbol table

<code>server/indexer/workerPool.js</code>	~167	Worker thread pool for parallel parsing
<code>server/indexer/workerTask.js</code>	~19	Worker entry point (calls fileSymbolParser)
<code>server/indexer/fileSymbolParser.js</code>	~37	Heuristic regex parser for Sub/Type/Class
<code>server/indexer/extractMethod.js</code>	~63	Extract Method refactoring
<code>server/logger.js</code>	~66	Async file logger with bounded queue
<code>scripts/bundle.js</code>	~80	esbuild bundler configuration

Appendix B: Data Flow Diagrams

B.1 Project Opening Flow

User opens .b4a file



`detectPlatformFromPath` → "b4a"



`parseProjectFile` → `allowedLibraries`, `allowedModuleFiles`



`getPlatformSettings` → discovers INI path



`loadPlatformIni` → `LibrariesFolder`, `AdditionalLibs`, `SharedModules`



`resolveAllowedLibraries` → XML files, b4xlib files



— `xmlLibraries.replaceXmlFiles(xmlFiles)`



— Extract .bas from .b4xlib archives



— `workspaceClasses.replaceReferenceModules(extractedBas)`



— `workspaceClasses.refresh(workspaceModules)`



— `commonClass.syncFrom(xmlLibraries)`



— `primitiveTypes.syncFrom(xmlLibraries)`





B.2 Completion Request Flow

User types "btn." in a .bas file



B.3 Rename Flow

User places cursor on "CalculateTotal" and presses F2

↓

LSP server: onRenameRequest

↓

findDefinition("CalculateTotal")

→ { name: "CalculateTotal", file: "Utils.bas", line: 15 }

↓

Collect candidate file paths:

- All open documents in DocumentManager
- All files from global.getByExactName("CalculateTotal")
- The definition file itself

↓

Process 4 files concurrently:

For each file:

1. Read file content
2. Find all \bCalculateTotal\b matches
3. Skip matches inside quoted strings (isInQuotedString)
4. Skip matches in comment lines (isCommentLine)
5. Apply case preservation (preserveCase)
 - CALCULATETOTAL → CALCULATEDISCOUNT
 - CalculateTotal → CalculateDiscount
 - calculatetotal → calculatediscount

↓

Return WorkspaceEdit with changes across all files

↓

VS Code applies rename across project

Appendix C: Configuration Reference

VS Code Settings

Setting	Type	Default	Purpose
---------	------	---------	---------

<code>b4xIntellisense.b4aIniPath</code>	string	""	Path to B4A's b4xV5.ini
<code>b4xIntellisense.b4jIniPath</code>	string	""	Path to B4J's b4xV5.ini
<code>b4xIntellisense.b4iIniPath</code>	string	""	Path to B4i's b4xV5.ini
<code>b4xIntellisense.b4rIniPath</code>	string	""	Path to B4R's b4xV5.ini
<code>b4xIntellisense.debug</code>	boolean	false	Enable file logging
<code>b4xIntellisense.autoOpenProjectFolderOnOpen</code>	boolean	false	Auto-add project folder to workspace

LSP Server Configuration

The LSP server receives its configuration through `initializationOptions`:

```
{
  "projectRoot": "/path/to/b4x/project"
}
```

Snapshot File

The `.b4x-index.json` file is stored in the project root and contains:

```
{
  "symbolStore": [[1, {"kind": "sub", "name": "Activity_Create", "line": 0,
"file": "/path/Main.bas", "_id": 1}], ...],
  "fileToIds": ["/path/Main.bas", [1, 2, 3]], ...],
  "lastId": 42
}
```

End of Research Paper